

SC2005 Lab 1 — Exploring xv6 and Adding a System Call

What this lab is about

xv6 is a small teaching operating system for RISC-V. In this lab, you will use its shell, inspect how user programs become part of the xv6 file system, observe processes, add one system call, and implement communication through pipes.

This is an AI-assisted lab. You may use the course Gemini Notebook, which contains the Lab 1 source tree and the xv6 book. The notebook should help you inspect and explain the supplied sources; it does not replace building, running, and checking your own work.

By the end of the lab, you should be able to:

1. Navigate a terminal, run a command, interrupt it, and distinguish the host shell from the xv6 shell.
2. Build and boot the course version of xv6 and exit QEMU correctly.
3. Explain how a C file under `user/` becomes a command available inside xv6.
4. Use `fork()`, process IDs, and process states to explain a simple process experiment.
5. Use source-code reference to trace a system call across user and kernel space, then add `getproccount()` across that boundary.
6. Explain pipe endpoints and descriptor lifetime, then complete a two-pipe parent-child program.

How to use Gemini Notebook in this lab

Use the following loop for each task:

1. **Formulate expectations:** write down what you expect to happen before asking AI or running the program.
2. **Research:** ask Gemini Notebook a focused question about the supplied source. Request file names, function names, and evidence.
3. **Verify in source:** open the cited file and check that the answer matches this Lab 1 branch.
4. **Verify by execution:** build and run a small test whose output can confirm or reject the claim.
5. **Explain:** state the result in your own words.

A useful prompt pattern for the step 2 above is:

In the supplied Lab 1 xv6 source, explain **[specific behavior]**. Identify the relevant files and functions, quote only the smallest necessary code fragment, and give me one hypothesis I can test. Do not write the complete exercise solution.

If the AI answer conflicts with the supplied code or with your observed output, trust neither blindly. Re-check the relevant function, reduce the problem to a smaller test, and ask a TA when the conflict remains.

At every checkpoint, be ready to show:

- your hypothesis;
- the source evidence you checked;
- the command and output used for verification; and
- a one- or two-sentence explanation in your own words.

Before you begin: two shells, two prompts

You will interact with two different shells:

Label used in this guide	Where the command runs	Examples
host\$	The Linux lab machine or VM	pwd, cd, make qemu
xv6\$	The xv6 shell running inside QEMU	ls, cat README, forktest

The labels `host$` and `xv6$` are explanatory labels. **Do not type the label itself.** Type only the command after it. `host` and `xv6` will not be printed in the shell. They are for clarification on which terminal to run.

Commands available on the Linux host are not automatically available in xv6 (for example, tab auto-completion). When unsure where you are, look at what you launched and what output appeared most recently.

Part 0 — Terminal essentials

This short warm-up teaches only what you need for the lab.

0.1 Navigate and inspect

Command references you can use in the terminal :

Input	Meaning
<code>pwd</code>	Print the current working directory.
<code>ls</code>	List entries in the current directory.
<code>cd directory</code>	Change the shell's working directory to "directory".
<code>cd ..</code>	Move to the parent directory.
<code>cat file</code>	Print a text file to the terminal.
Tab	Ask the shell to complete a command or path.
Up Arrow	Recall a previous command.

A command line normally begins with a program or shell command and is followed by its arguments. In `cat README.md`, `cat` is the program, and `README.md` is an argument.

If your repository directory has a different name, use the file name shown by `ls`.

At the host prompt, try:

1. Check out the current working directory
2. Move to the repository folder you just cloned
3. Check out the files in the folder
4. Print the `README.md` content to the terminal

0.2 Run and interrupt a foreground program

Run a command that waits for 30 seconds and returns:

```
host$ sleep 30
```

While it is running, press `Ctrl-C`. The terminal may display `^C` and return to the host prompt. `Ctrl-C` asks the host shell to interrupt its foreground program; it is not text that you type into the command.

Checkpoint 0: explain before continuing

You are ready to continue when you can:

- use `pwd`, `ls`, and `cd` to enter the repository
- identify the command and argument in `cat README.md`
- stop `sleep 30` with `Ctrl-C`

Part 1 — Build, boot, and explore xv6

1.1 Obtain the correct source

Clone the Lab 1 source from the public course registry.

```
host$ git clone https://github.com/CPS-research-group/ntu-sc2005-xv6.git
```

```
host$ cd ntu-sc2005-xv6
```

```
host$ git checkout lab1
```

```
host$ git branch --show-current
```

Hint: Ask Gemini Notebook about what the above commands do; you will use them often.

1.2 Build and boot

For the first build, start from a clean tree. First confirm that you are at the repository root—the directory containing `Makefile`—and then run:

```
host$ pwd
```

```
host$ ls Makefile
```

```
host$ make clean
```

```
host$ make qemu
```

The initial `make clean` removes compiled files, generated system-call stubs, and an old file-system image that may have been left by an earlier build. On a newly extracted or cloned tree, it may have little or nothing to remove. You do not need to run it before every edit: during the lab, `make qemu` should rebuild changed components incrementally. Clean again only after switching to a different branch or commit, or when a build appears to use stale generated files.

`make qemu` compiles the RISC-V kernel and selected user programs, constructs the xv6 file-system image, and starts QEMU. A successful boot eventually gives you an xv6 prompt.

At the xv6 prompt, try:

```
xv6$ ls
```

```
xv6$ cat README
```

```
xv6$ mkdir lab1tmp
```

```
xv6$ cd lab1tmp
```

```
xv6$ cd ..
```

```
xv6$ ls
```

Notice that the xv6 file is named `README`, whereas the host repository also has `README.md`. The host and xv6 do not share one current directory or one shell.

While QEMU is running, use `Ctrl-a x` to exit QEMU: hold `Ctrl` and press `a`, release both, and then press `x`. This is different from `Ctrl-C`.

Exit QEMU with `Ctrl-a x` and confirm that the host prompt returns.

Ask Gemini Notebook

Using the supplied source, explain what the `qemu` and `fs.img` targets in `Makefile` produce. Which list determines the user programs copied into the xv6 file-system image?

Check the answer against `Makefile` before continuing.

Checkpoint 1: explain before continuing

You are ready when you can:

- boot to the xv6 prompt and exit back to the host
- state whether `make qemu` runs on the host or inside xv6
- point to the `UPROGS` list in `Makefile` and explain its purpose

Part 2 — User programs are built into the xv6 image

The existence of `user/program.c` does not make `program` an xv6 command. The source must be compiled, linked with the xv6 user library, and included in `fs.img` through `UPROGS`.

2.1 Establish a baseline

Boot xv6 and run the programs already included in the image:

```
xv6$ forktest
```

```
xv6$ usertests
```

`forktest` should end with `fork test OK`. `usertests` runs a larger test suite and should finish without reporting a failed test. If it fails before you edit anything, preserve the output and ask Gemini Notebook for help. If the problem persists, ask a TA.

Now try:

```
xv6$ pingpong
```

In the supplied branch, `user/pingpong.c` exists, but `_pingpong` is initially absent from `UPROGS`. Predict the result before running it.

Ask Gemini Notebook

Trace only the build path for `user/pingpong.c` in this branch: source file, `UPROGS` entry, generated user binary, and `fs.img`. Why can a source file exist while the xv6 shell still reports that execution failed?

Then verify each named item in `Makefile`.

2.2 Add `pingpong` to the image

Exit QEMU. Open `Makefile`, locate `UPROGS`, and add the entry for `_pingpong` using the same pattern as the existing entries. Rebuild and boot:

```
host$ make qemu
```

Run `pingpong` again inside xv6.

Checkpoint 2: explain before continuing

You are ready when you can:

- explain why `pingpong` was initially unavailable

- show the **UPROGS** change and successful execution

Part 3 — Observe processes and a zombie state

A process that has exited remains in the `ZOMBIE` state until its parent collects its status with `wait()`. The supplied `user/zombie.c` creates this situation, but its original observation window is short.

3.1 Read, predict, and make the state observable

Read `user/zombie.c` and `kernel/proc.c`. In particular, locate:

- the return convention of `fork()`
- `exit()` and the assignment to `ZOMBIE`
- `wait()` and the point at which a zombie slot is freed
- `procdump()`, which prints process states.

For easier observation, temporarily change `sleep(5)` in `user/zombie.c` to `sleep(100)`. In this xv6 branch, the argument is a number of clock ticks, not seconds.

Rebuild, boot, and run:

```
xv6$ zombie
```

While the program is still running, press `Ctrl-p`. This console key invokes `procdump()` and prints the process list. Look for a `zombie` entry. After the parent exits and the shell prompt returns, press `Ctrl-p` again and compare the listings.

Ask Gemini Notebook

In the supplied `zombie.c` and `proc.c`, explain the exact event order that makes the child a zombie. What later action allows its process-table slot to return to `UNUSED`? Cite the relevant functions.

Checkpoint 3: explain before continuing

You are ready when you can:

- explain the different values returned by `fork()` in the parent and child
- show a `Ctrl-p` process listing containing a zombie
- distinguish “the process has stopped running” from “its process-table entry has been reaped.”

Part 4 — Trace and add `getproccount()` system call

A system call is a controlled transition from a user program into privileged kernel code. Before adding one, trace the existing `getpid()` system call in this exact branch.

4.1 Ask Gemini Notebook to trace `getpid()`

Begin with a source-navigation task by asking Gemini Notebook:

In the uploaded Lab 1 sources, trace the existing `getpid()` system call end to end. Include both (1) the build-time path that creates its user-space call stub and (2) the runtime path from a user C call, through the trap and numbered dispatcher, to the kernel code that obtains the PID and returns it. Give an ordered table with the exact original file path, function or symbol, and line range for every stage. Mark generated files separately and cite the generator or Makefile rule instead of inventing lines for a file that is not uploaded. Cite the relevant xv6 book section.

Record the answer in a trace table of your own:

Order	Build time or runtime?	Exact path and line(s)	Function or symbol	Role in the call path
1				
2				
...				

4.2 Verify the trace in the source

Open every cited location and check the cited lines. Correct the table when the path, symbol, line range, or ordering does not match the supplied branch. Your verified trace must account for all of the following milestones:

- the user-visible declaration
- the source and build rule for the generated user stub
- the system-call number placed in a register before `ecall`
- the user trap and common system-call dispatcher
- the numbered dispatch-table entry and kernel handler
- the kernel expression that obtains the PID and the register used for the return value

At this point, also determine which source file is generated during a build and which source file should be edited instead.

4.3 Define the new call's contract

Implement this interface:

```
int getproccount(void);
```

For this lab, its result is defined as:

The number of process-table entries whose state is not **UNUSED** at the instant each entry is inspected. This includes **USED**, **SLEEPING**, **RUNNABLE**, **RUNNING**, and **ZOMBIE** entries.

4.4 Ask Gemini Notebook to map the required changes

Now use the verified **getpid()** trace to plan the new call:

Using the verified **getpid()** path and only the uploaded Lab 1 sources, map what is required to add a system call named **getproccount** plus a user program named **proccount** that invokes it. For every existing location, give the exact original file path, nearby function or symbol, and line range to inspect. Classify each item as **inspect only**, **edit**, **create**, or **generated—do not edit**, and explain why. Separate system-call registration from the process-counting implementation. Include the user program's inclusion in the xv6 file-system image. Do not provide C, assembly, or a patch.

Verify every cited location, then make your own implementation checklist:

Action	Exact path, line(s), and nearby symbol	Intended responsibility	Verified in source?
Inspect / edit / create / generated			
...			

4.5 Implement the verified plan

Use **getpid()** as the registration example, and implement the **getproccount** system call. Also, write a user program **proccount** that triggers the system call and prints the

4.6 Verify behavior, not just compilation

After rebuilding, run:

```
xv6$ proccount
```

Invoke **proccount**, and compare with the output of **Ctrl-p**. Explain why the output differs between the two.

When something fails during the implementation, preserve the output and ask Gemini Notebook for help. If the problem persists, ask a TA.

Checkpoint 4: explain before continuing

You are ready when you can:

- show a verified `getpid()` trace with exact files, functions or symbols, and line ranges
- distinguish locations that are inspected, edited, created, or generated when adding `getproccount()`
- trace the new call from its user declaration and stub generator to its kernel handler
- explain why the system-call number must match across the stub and dispatch table
- explain why the generic trap path is part of the trace but does not need a per-call edit
- show a working `proccount` user program and defend its output as plausible
- explain why the output of `proccount` and `Ctrl-p` differs

Part 5 — Communicate with two pipes

Pipes are a fundamental inter-process communication (IPC) mechanism in Unix-like systems, including xv6. They allow data to flow from one process to another — for example, the output of one command can become the input of another.

`pipe(int p[2])` creates one unidirectional byte stream:

- `p[0]` is the read descriptor;
- `p[1]` is the write descriptor.

After `fork()`, parent and child descriptor tables refer to the same underlying open-file objects. xv6 increments reference counts in `filedup()`. Closing one process's descriptor removes that reference; however, it does not close the endpoint for every process while another reference remains.

5.1 Connect the abstraction to the source

Inspect:

- `user/pingpong.c` for user-level `pipe()`, `fork()`, `read()`, and `write()` calls
- `kernel/sysfile.c` for the system-call side of `pipe()`
- `kernel/pipe.c` for buffering, blocking, EOF, and error behavior
- `kernel/proc.c` for descriptor duplication during `fork()`
- `kernel/file.c` for `filedup()` and `fileclose()`

Draw the descriptors immediately after `fork()` and cross out the unused endpoint in each process. Close unused endpoints promptly. Keeping an unused write descriptor open can prevent a reader from observing EOF.

5.2 Design and implement `two_pipes.c` with Gemini Notebook

Complete `user/two_pipes.c` using two pipes: the parent sends `ping`, the child prints `<child-pid>: received ping` and replies with `pong`, and the parent prints `<parent-pid>: received pong`. The program must check failed operations, handle received bytes safely, close descriptors it no longer needs, wait for the child, and exit with a meaningful status.

Before coding, ask Gemini Notebook to help you make an endpoint and operation plan:

Inspect `user/two_pipes.c` and the pipe-related code in the uploaded sources. Help me plan—without writing the complete code—a two-pipe exchange between the parent and child created by its single `fork()`. Name all four endpoints, state which process keeps or closes each one, and order the `read`, `write`, `close`, and

`wait` operations. Include the checks needed for errors and safe string handling.
Cite exact source paths and lines.

Turn the response into a small plan and verify its citations in the uploaded source. Then implement it, add `_two_pipes` to `UPROGS`, rebuild, and run it several times. PIDs may vary, but the child must receive `ping` before the parent receives `pong`.

If the program does not build, hangs, or prints unexpected output, describe the exact error or behavior to Gemini Notebook and ask for diagnostic guidance:

My `two_pipes` program [does not build / hangs / prints unexpected output]. The exact error or behavior is: _____. Give me a step-by-step debugging checklist based only on the uploaded sources. Focus on endpoint ownership, operation order, return values, message length, and references that may still be open. Do not provide a complete solution or patch.

Check each suggestion against the source, change one thing at a time, and rerun the program.

5.3 Investigate endpoint closure

Use Gemini Notebook and `kernel/pipe.c` to predict what `read()` or `write()` does when one end of a pipe is closed prematurely. Ask Gemini Notebook to cite the exact conditions in `kernel/pipe.c` and trace how `fork()` and `close()` change the relevant reference counts through `kernel/proc.c` and `kernel/file.c`.

Test one prediction with a small temporary change to `two_pipes.c`, compare the result with your prediction, and then restore the working implementation.

Checkpoint 5: explain before finishing

You are ready when you can:

- label the read and write descriptor for each pipe
- explain why full-duplex request–reply communication uses two pipes here
- explain why each process closes unused endpoints
- distinguish closing one descriptor reference from closing the last reference to a pipe endpoint

Final checkpoint before leaving

Before leaving, you should be able to answer without AI:

1. Why is adding `user/program.c` alone insufficient to make `program` available in xv6?
2. What does the generated system-call stub place in register `a7`, and what instruction enters the kernel?
3. Why can `getproccount()` return different value from `Ctrl-p`?
4. Why does closing one post-`fork()` descriptor not necessarily close the underlying pipe endpoint?
5. What result distinguishes pipe EOF from a pipe write with no readers?

References

- MIT Missing Semester, Course Overview + Introduction to the Shell.
- MIT Missing Semester, Command-line Environment, especially the discussion of Ctrl-C and signals.
- Russ Cox, Frans Kaashoek, and Robert Morris, [*xv6: a simple, Unix-like teaching operating system*](#), especially §§1.1–1.3, §2.2, §4.3, and §9.3.